Compression Algorithms for Block Model Project — Polished Visual Report (Aug 2025)

Executive Summary

This project requires compressing large 3D block models into rectangular sub-blocks. Scores are based on speed and compression ratio (fewer blocks). The main strategy combines uniform parent detection, 2D greedy rectangle tiling, and 3D extrusion merges across slices. Data must be streamed, processing ≤ parent_z slices at a time.

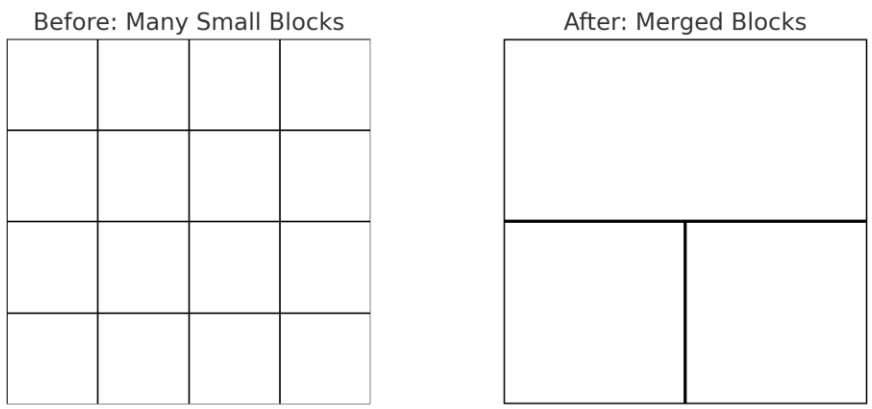

Figure 1: Block compression – many small blocks merged into fewer large blocks.

Compression Families at a Glance

• Run-Length Encoding (RLE): simple, streaming-friendly, weak on noisy data.
• Dictionary methods (LZ77/LZW/DEFLATE/Zstd/Brotli): byte-focused, not optimal for block count.
• Entropy coding (Huffman/Arithmetic): saves bits but not blocks.
• Tree/Graph (Octree/Quadtree): effective on uniform regions, hierarchical output.
• Segmentation/Connected Components: handles irregular shapes, slower.

Algorithms for This Project

1. Per-slice RLE + maximal rectangles: efficiently tile slices using runs and histograms.
2. 3D extrusion: extend rectangles across slices if footprints and labels match.
3. Uniform parent detection: collapse parent block if all cells share the same label.
4. Optional: octree shortcuts or connected-component tiling for irregular cases.

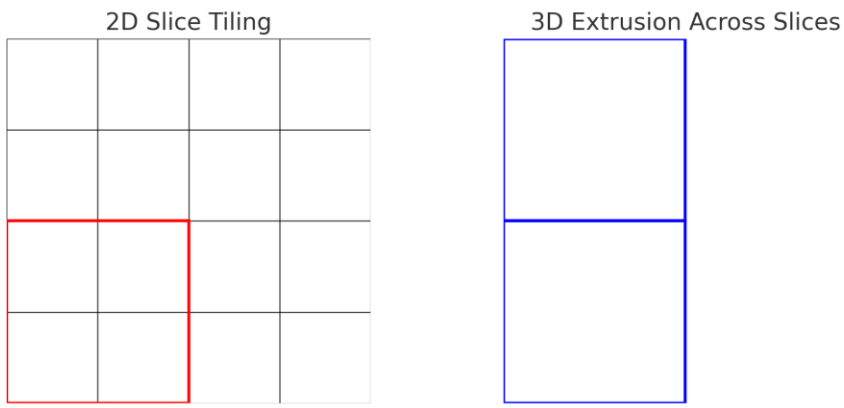

Figure 2: Left – 2D rectangle tiling on a slice. Right – 3D extrusion across slices.

Complexity & Performance

Aspect	Details	Notes
Memory	O(parent_x·parent_y·parent_z) per parent window	Bounded by parent block size
2D Tiling	O(Rectangles × X × Y) per slice	Greedy/histogram-based tiling
3D Merge	O(#rectangles)	Merge across slices
Parallelism	Independent per-parent block	GPU-friendly row-wise scans

Heuristics

1. Process frequent labels first.
2. Merge adjacent runs.
3. Hash footprints for z-merges.
4. Flush early when merge fails.
5. Iterate in multiples of parent size for cache efficiency.
6. Delay small 1×1 tiles until later.

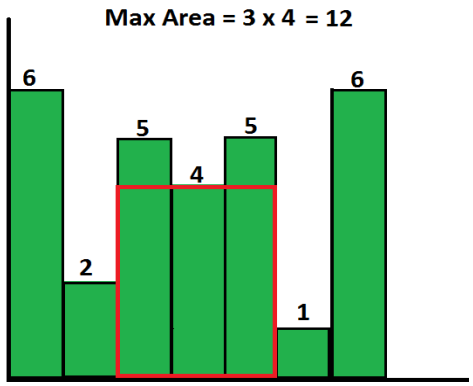

Figure 3: Largest rectangle in histogram (source: Coderworld Blog).

Suggested System Design

Pipeline:

1. Input streaming (header, tags, slices).
2. Parent-block scheduler in raster order.
3. Compression steps: uniformity test, 2D tiling, 3D extrusion.
4. Output writer: emit x,y,z,size,label.
5. Parallelism: threads per parent-block.

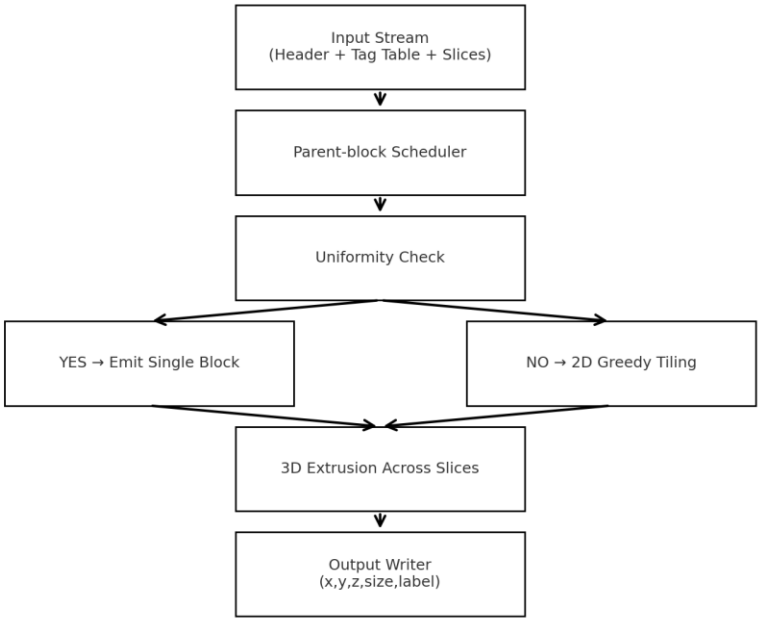

Figure 4: Compression pipeline flowchart from input to output.

Evaluation Plan

Metric	How to Measure	Tools
Correctness	Unit tests on small grids	Custom test cases
Compression Ratio	Output blocks / Input blocks	Parser & counter script
Speed	Time per million voxels	Timer libraries
Heuristic Impact	Ablation studies	Toggle heuristics

Additional Research Materials

1. Divide & Conquer + Greedy Block Merging (GitHub): practical baseline implementation. [Source: github.com/jy-tai/Block-Model-Compression-Algorithm]

2. Voxel Model Compression — TU Dresden: compared Cuboid Extraction, Sparse Voxel Octrees (SVOs), and RLE. Takeaway: RLE + Octree hybrids efficient. [Source: eisenwave.github.io]

3. Geological Block Merging Framework (JOSIS 2020): reduces fragmentation, produces balanced aspect ratios. [Source: arxiv.org/abs/2001.04023]

4. VoxelDNN: deep learning + arithmetic coding, ~28% better than MPEG G-PCC. [Sources: arxiv.org/abs/2011.14700, digicosme.cnrs.fr]

5. Voxel Scene Compression with DAGs: compact for sparse data, resolution-limited. [Source: timothykol.com]

6. Deep Implicit Volume Compression (2020): neural approach, 66% bitrate reduction, high compute cost. [Source: arxiv.org/abs/2005.08877]

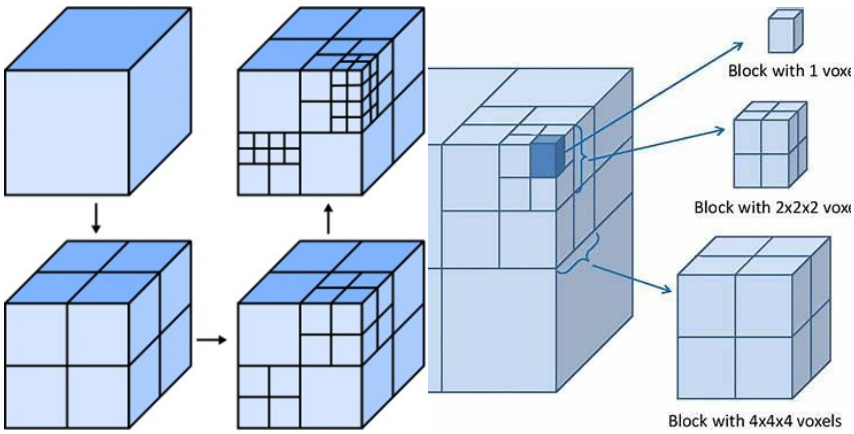

Figure 5: Octree partitioning of voxels (ResearchGate).

Comparison Summary

• GitHub (Divide & Conquer/Greedy): practical baseline.
• TU Dresden (SVO/RLE): validates RLE and octrees.
• JOSIS Geological Merging: reduces fragmentation, parallelizable.
• VoxelDNN: strong compression but complex.
• DAG Compression: compact representation, limited scope.
• Deep Implicit Volumes: high compression, high compute cost.